AISSMS
COLLEGE OF ENGINEERING
Approved by AICTE, New Delhi, Recognized by
Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University
and recognized 2(f) and 12(B) by UGC (Id.No. PU / PN/ Engg. / 093 (1992)
Accredited by NAAC with 'A+' Grade

Department of Computer Engineering

Code2UML Mini Project Report

(Object Oriented Modeling And Design)

Submitted in partial fulfilment of the requirements for the degree

of

BACHELOR OF ENGINEERING

In

COMUTER ENGINEERING

Submitted By

Name of the student:

Shravani Lokhande

Roll No.: 24CO308

Under the Guidance of

Prof. A. J. Kadam

Academic Year: 2026-21(Term-I)

ABSTRACT

Software systems are often represented using Unified Modeling Language (UML) diagrams to communicate structure and behavior. Creating these diagrams manually from source code can require additional time and may introduce inconsistencies between implementation and documentation.

Code2UML is a Java-based prototype that explores automated generation of UML diagrams from Java source code. The system accepts a folder containing Java source files, analyzes the source structure, and produces graphical UML output. JavaParser is used for source-code parsing and PlantUML is used for rendering diagram descriptions into PNG images.

The project demonstrates multiple UML diagram types using a Student Course Registration scenario. The implemented demonstration includes a Class Diagram, Sequence Diagram, Activity Diagram, and Component Diagram. The Class Diagram represents classes and their relationships, the Sequence Diagram represents method interactions, the Activity Diagram represents registration workflow and decision flow, and the Component Diagram represents major system components.

The project is intended as an academic prototype demonstrating how source-code analysis can be connected with UML visualization. It provides a foundation that can later be extended with more advanced source-code analysis, a graphical user interface, additional UML diagram types, and improved inference of behavior from arbitrary Java programs.

Keywords

UML, Java Source Code, JavaParser, PlantUML, Code Analysis, Class Diagram, Sequence Diagram, Activity Diagram, Component Diagram.

1. INTRODUCTION

1.1 Background

Unified Modeling Language (UML) is widely used to visualize software systems before and during implementation. Different UML diagrams provide different views of a system. Structural diagrams describe elements such as classes and components, while behavioral diagrams describe interactions and workflows.

In a conventional development process, developers or students often create UML diagrams manually. When source code changes, the corresponding diagrams may also need to be updated. An automated or semi-automated approach can reduce repetitive work by extracting information directly from source code.

1.2 Project Idea

Code2UML is designed around the idea of converting Java source code into UML diagram representations. The prototype uses JavaParser to inspect Java source files and PlantUML to render diagram descriptions. Maven is used to build and execute the Java project.

1.3 Demonstration Scenario

The selected demonstration scenario is Student Course Registration. A Student attempts to register for a Course. The registration method checks course availability. If the course is available, the student is added to the course and a success message is displayed; otherwise, the system displays that the course is full.

Student.java

```
public class Student {
   public void registerCourse(Course course) {
      if (course.isAvailable()) {
         course.addStudent(this);
         System.out.println("Course registered successfully");
      } else {
         System.out.println("Course is full");
      }
   }
}
```

2. PROBLEM STATEMENT AND MOTIVATION

2.1 Problem Statement

Develop a prototype that accepts Java source code as input, analyzes relevant source-code information, and generates UML diagram output that can be used to understand the structure and behavior of the source program.

2.2 Motivation

Manual UML creation requires the user to inspect source code, identify classes and methods, understand method interactions, and draw diagrams. For educational projects, this process can be repetitive. Code2UML investigates whether commonly available source parsing and diagram-rendering technologies can be connected to automate part of this process.

2.3 Need for the System

- Reduce repetitive manual UML drawing for supported source-code patterns.
- Provide a visual representation of Java source code.
- Demonstrate the use of source-code parsing in a practical software engineering application.
- Generate diagram images that can be used for documentation and demonstrations.
- Provide a foundation for future extension toward additional UML diagram types.

2.4 Problem Scope

The current project is a prototype rather than a complete Java reverse-engineering platform. Its strongest demonstrated capability is source analysis for the selected class and sequence scenarios. Activity and Component diagrams are currently generated using defined scenario rules. More advanced automatic inference can be added in future versions.

3. OBJECTIVES AND SCOPE

3.1 Objectives

1. Accept a Java source-code folder as input.
2. Parse Java source files using JavaParser.
3. Identify classes and methods needed for UML generation.
4. Generate PlantUML representations.
5. Render UML representations as PNG images using PlantUML.
6. Demonstrate multiple UML diagram types using a common scenario.
7. Provide a command-line workflow suitable for academic demonstration.

3.2 Scope

The project focuses on Java source code and a command-line based generation workflow. The demonstration uses Student.java and Course.java. The generator project itself is maintained separately from the input test folder so that source examples can be tested without changing the generator implementation.

3.3 Expected Outcome

For a supported input scenario, the user can invoke the generator with a diagram type and input/output parameters. The system produces a PNG diagram in the input directory.

3.4 Future Scope

- Support additional Java language constructs.
- Automatically infer Activity and State diagrams from source-code control flow and state changes.
- Add a web or desktop user interface.
- Support additional UML diagram types.
- Provide SVG/PDF export and interactive diagram viewing.
- Improve handling of method overloading, inheritance, interfaces, and nested classes.

4. EXISTING SYSTEM AND PROPOSED SYSTEM

4.1 Existing / Conventional Approach

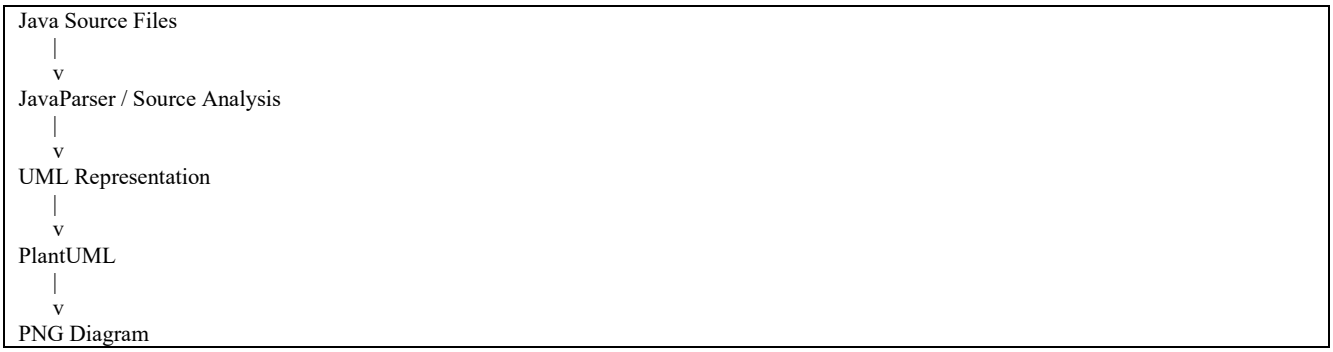
In a conventional approach, the developer reads source code and manually prepares UML diagrams using a diagramming application or drawing tool. The process requires separate work for each diagram and may need repeated updates when the source code changes.

4.2 Proposed System

The proposed Code2UML prototype connects source-code analysis with UML rendering. The user provides a Java source-code folder and selects the required generation command. The source code is analyzed and a PlantUML representation is created for the supported diagram type. PlantUML then generates the final PNG image.

Aspect	Conventional Approach	Code2UML Prototype
Input	Manual interpretation	Java source-code folder
Diagram creation	Manual drawing	Generated PlantUML representation
Rendering	Diagram tool	PlantUML
Source analysis	Developer inspection	JavaParser for supported cases
Output	Manually saved diagram	PNG diagram
Demonstration	Separate manual steps	Command-line workflow

4.3 Proposed Workflow



5. SYSTEM REQUIREMENTS AND TECHNOLOGIES

5.1 Hardware Requirements

- Computer or laptop capable of running Java and Maven.
- Minimum 4 GB RAM recommended for the academic prototype.
- Sufficient disk space for Java, Maven dependencies, project source files, and generated images.

5.2 Software Requirements

Technology / Tool	Purpose
Java	Implementation language
JavaParser	Parsing and analysis of Java source code
PlantUML	Rendering UML descriptions into diagrams
Apache Maven	Build and dependency management
Windows Command Prompt	Command-line execution and demonstration
VS Code / Notepad	Source-code editing

5.3 Project Environment

The project was developed and tested in a Windows environment. The local environment used Java 24.0.2 and Apache Maven 3.9.9. The generator was built successfully using Maven.

5.4 Directory Structure

```
D:\stqa
|
+-- uml_diagram_generator_using_java
|     +-- src
|     +-- pom.xml
|     +-- ...
|
+-- student-course-test
      +-- Student.java
      +-- Course.java
      +-- generated PNG diagrams
```

6. SYSTEM ARCHITECTURE AND WORKING METHODOLOGY

6.1 Architecture



6.2 Working Steps

8. The user prepares Java source files in an input folder.
9. The generator reads the source folder.
10. JavaParser structures the Java source into parseable program elements.
11. The required generator identifies the information needed for the selected diagram.
12. PlantUML text is prepared.
13. PlantUML renders the text into a PNG image.
14. The generated image is saved in the input directory.

6.3 Separation of Input and Generator

The generator project and test source folder are kept separate. This allows the same generator to be tested with different Java examples without modifying the generator repository or the existing test cases.

7. IMPLEMENTATION

7.1 Class Diagram Generation

The Class Diagram generator analyzes Java source files to identify classes and their relevant members. The Student Course Registration example contains Student and Course classes. Their methods provide the basis for representing the class structure and relationship.

7.2 Sequence Diagram Generation

The Sequence Generator identifies methods and method calls. The original implementation was extended to recursively collect method-call expressions so calls inside control-flow blocks such as the if statement can be detected. For the demonstration, Student.registerCourse() is used as the starting method.

```
Student.registerCourse(Course course)
      |
      +--> course.isAvailable()
      |
      +--> course.addStudent(this)
```

7.3 Activity Diagram Generation

The Activity Diagram demonstration represents the registration workflow, including the decision on course availability. The current prototype uses a defined PlantUML activity representation for the selected Student Course Registration scenario.

7.4 Component Diagram Generation

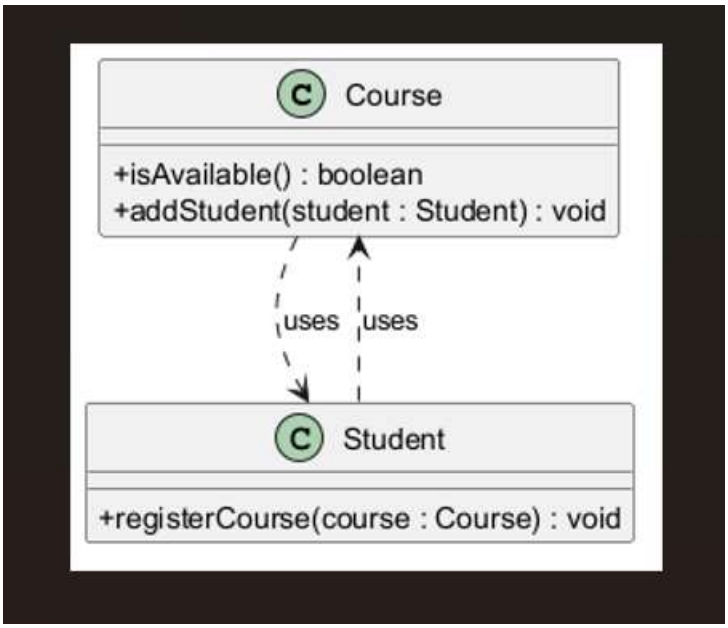
The Component Diagram demonstration represents Student and Course as major components and shows their interaction during registration. The current prototype uses a defined PlantUML representation for this scenario.

8. UML DIAGRAMS OF THE SYSTEM

8.1 Class Diagram

The Class Diagram represents the structural view of the Student Course Registration example. It contains Student and Course classes and their relevant operations.

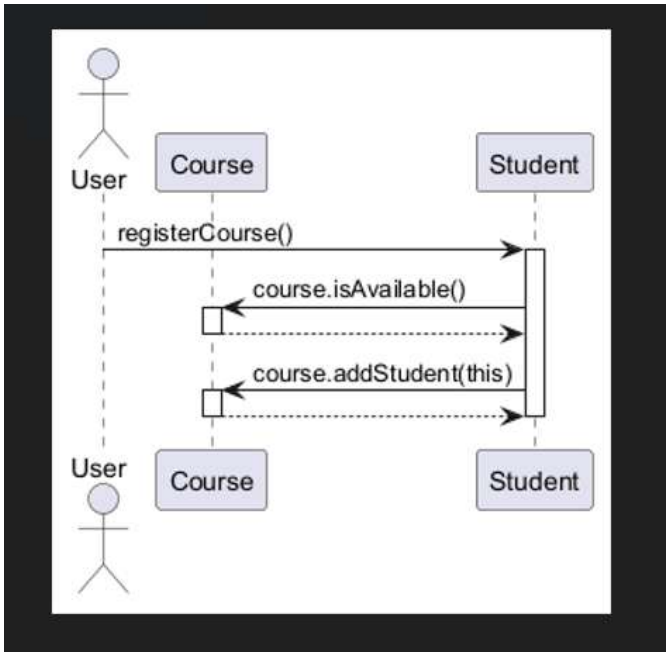
```
Student
-------------------------
+ registerCourse(Course)
-------------------------
        |
        | uses
        v
Course
-------------------------
+ isAvailable()
+ addStudent(Student)
-------------------------
```



8.2 Sequence Diagram

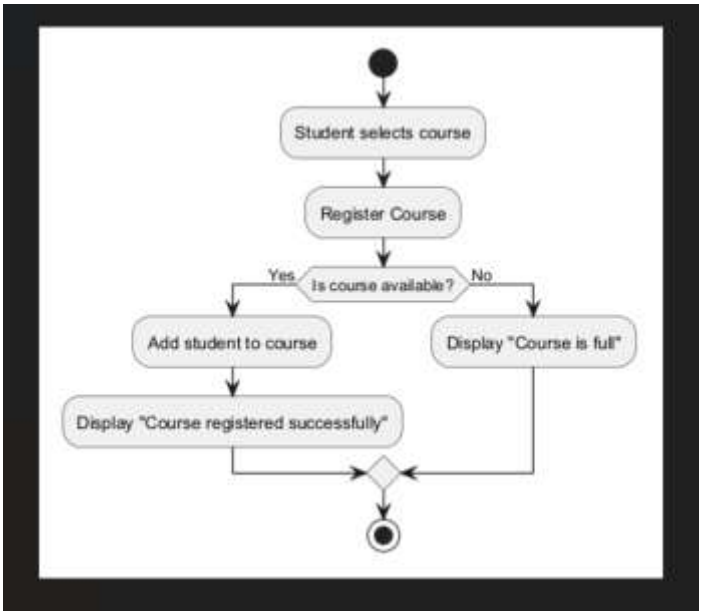
The Sequence Diagram represents the order of interactions beginning with Student.registerCourse(). The method checks course availability and, for the successful path, invokes Course.addStudent().

```
User -> Student : registerCourse()
Student -> Course : course.isAvailable()
Course --> Student
Student -> Course : course.addStudent(this)
Course --> Student
```



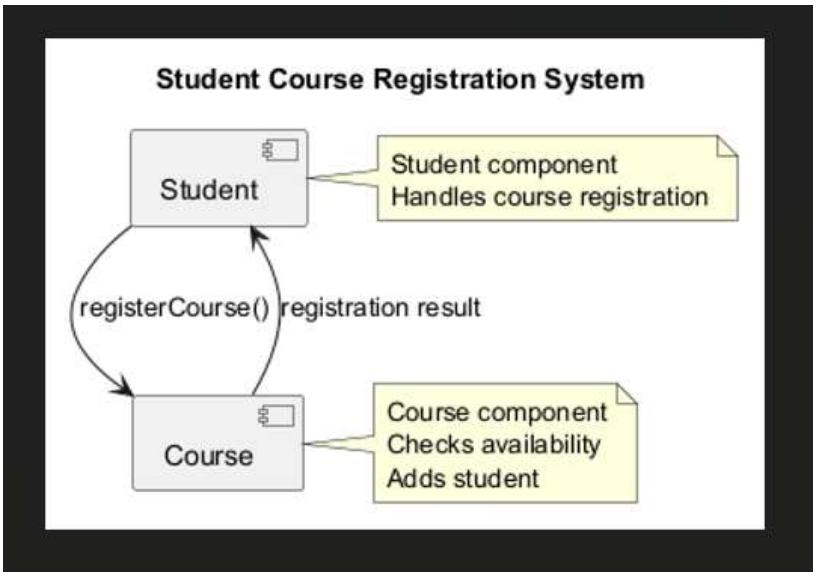
8.3 Activity Diagram

Start
|
Select Course
|
Register Course
|
[Course available?]
| Yes | No
v v
Add Student Course is full
|
Registration successful
|
End



8.4 Component Diagram

```
+-----------------------------------+
| Student Course Registration       |
|                                   |
|   [Student] ----> [Course]        |
|      registerCourse()             |
+-----------------------------------+
```



9. TESTING AND RESULTS

9.1 Test Scenario

The Student Course Registration scenario was used to verify the generator. The source folder contains Student.java and Course.java. The generator is executed from the UML generator project directory.

9.2 Build Test

```
cd /d D:\stqa\uml_diagram_generator_using_java
mvn clean package

Expected result:
BUILD SUCCESS
```

9.3 Diagram Generation Tests

Test	Command / Input	Expected Result
Class	class + student-course-test + student-course	Class PNG generated
Sequence	seq + Student + registerCourse + student-course-sequence	Sequence PNG generated
Activity	activity + student-course-test + student-course-activity	Activity PNG generated
Component	component + student-course-test + student-course-component	Component PNG generated

9.4 Result

The project was successfully built with Maven after adapting the Sequence Generator to the JavaParser API used by the project. The Class and Sequence demonstrations were tested with the Student Course Registration source files, and the diagram output was generated as PNG files. Activity and Component diagrams were generated using the defined scenario representations.

9.5 Sample Output Location

```
D:\stqa\student-course-test\
   student-course.png
   student-course-sequence.png
   student-course-activity.png
   student-course-component.png
```

10. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

10.1 Advantages

- Reduces manual effort for supported UML generation tasks.
- Works directly with Java source-code files.
- Uses open-source technologies.
- Produces visual PNG output suitable for documentation.
- Separates source-code examples from the generator project.
- Provides a clear command-line demonstration workflow.

10.2 Limitations

- The current prototype does not perform complete semantic analysis of arbitrary Java programs.
- Complex Java constructs may require additional parsing logic.
- Method overloading and advanced type resolution are not fully handled.
- The Activity Diagram implementation is currently scenario-based rather than a complete control-flow reverse-engineering engine.
- The Component Diagram implementation is currently scenario-based.
- The prototype uses a command-line interface rather than a complete graphical/web interface.

10.3 Future Scope

- Develop a web interface for uploading Java projects.
- Automatically infer Activity Diagrams from if, switch, loop, and return statements.
- Automatically infer State Diagrams from enums, state variables, and transitions.
- Support inheritance, interfaces, packages, and dependencies.
- Generate SVG, PDF, and interactive UML outputs.
- Add project-wide analysis and navigation.
- Add error reporting for unsupported source-code constructs.

11. CONCLUSION

Code2UML demonstrates a practical approach to connecting Java source-code analysis with UML visualization. The project uses JavaParser for source-code parsing and PlantUML for diagram rendering. The Student Course Registration scenario provides a simple and understandable test case for demonstrating multiple UML views.

The Class Diagram provides a structural representation of Student and Course. The Sequence Diagram shows the method interaction beginning with Student.registerCourse(). The Activity Diagram represents the registration workflow and its availability decision. The Component Diagram provides a high-level representation of the participating components.

The project provides a useful academic foundation for further work in software engineering automation and source-code visualization. Its modular design allows additional diagram generators and more advanced source-code analysis to be introduced in future versions.